

Annotation Guidelines for Log Anomaly Labeling and Parsing-Error Coding

Replication Package for “When Parsing Goes Wrong: Revisiting Log Anomaly Detection from Parsing to Deployment”

1. Purpose

These guidelines describe the manual annotation procedures used in the study *When Parsing Goes Wrong: Revisiting Log Anomaly Detection from Parsing to Deployment*. The annotation process has two purposes:

1. To assign anomaly labels to log entries or log sequences when ground-truth labels are not publicly available.
2. To code observed parsing-error cases according to the six-category parsing-error taxonomy used in RQ1 and RQ2.

The goal is to ensure that annotation decisions are consistent, reproducible, and grounded in observable log behavior rather than subjective preference or parser-specific implementation details.

2. Annotator Roles

The annotation process should involve four domain experts or trained annotators with experience in software development, system operation, log analysis, or reliability engineering.

Annotators are divided into two independent expert groups:

- **Group A:** two annotators
- **Group B:** two annotators

Each group independently reviews the assigned log cases and produces group-level labels. Disagreements between the two groups are resolved through adjudication involving all annotators.

3. Input Materials

Annotators should be provided with the following materials:

- Raw log messages
- Parsed templates generated by each parser
- Extracted variable fields, if available
- Timestamp, module, component, process, host, or session information, if available
- Dataset-level description and domain context
- Known anomaly labels for reference datasets, if available
- The six-category parsing-error taxonomy
- Representative examples of normal, anomalous, and parsing-error cases

Annotators should not rely only on the parsed template. The raw log message should always be treated as the primary source of truth.

4. Anomaly Labeling Guidelines

4.1 Annotation Unit

The primary annotation unit is the log sequence used for anomaly detection. Depending on dataset characteristics, a sequence may be constructed by session-based grouping or fixed event-count windows.

When entry-level annotation is required, each raw log entry should first be labeled as normal or anomalous. Sequence-level labels are then derived from entry-level labels.

4.2 Label Definitions

Each log entry or sequence should be assigned one of the following labels:

- **Normal:** The log reflects expected system behavior, routine operation, successful execution, regular state transition, or benign warning that does not indicate failure, security risk, control deviation, or abnormal system behavior.
- **Anomalous:** The log indicates failure, exception, timeout, access violation, control deviation, safety alarm, abnormal sensor state, data corruption, unexpected state transition, service interruption, or other behavior requiring operational attention.
- **Uncertain:** The annotators cannot confidently determine the label based on the available context. This label should be temporary and must be resolved during adjudication.

4.3 Normal Log Criteria

A log should usually be labeled as normal if it describes:

- Successful task, request, transaction, or workflow completion
- Expected state change or lifecycle event
- Routine heartbeat, polling, monitoring, synchronization, or status update
- Benign system information, debug message, or configuration message
- Warning messages that are expected and do not indicate operational failure
- Repeated messages known to be part of normal execution

4.4 Anomalous Log Criteria

A log should usually be labeled as anomalous if it describes:

- Explicit failure, error, exception, timeout, crash, abort, or fatal event
- Failed authentication, unauthorized access, policy violation, or security alert
- Hardware, sensor, actuator, robot, controller, or communication fault
- Data corruption, checksum failure, missing file, invalid image, or invalid payload
- Control-loop deviation, safety override, emergency stop, abnormal retry pattern, or halted process
- Unexpected service unavailability, gateway failure, upstream failure, or queue blockage
- Rare operational state that domain experts identify as abnormal or requiring intervention

4.5 Sequence-Level Labeling Rule

A sequence should be labeled as anomalous if it contains at least one anomalous log entry. Otherwise, it should be labeled as normal.

If a sequence contains only uncertain entries, it should be marked for adjudication.

4.6 Anomaly Labeling Procedure

1. Each expert group independently reviews the assigned log entries or sequences.
2. Each group assigns one label: Normal, Anomalous, or Uncertain.
3. The two group-level labels are compared.
4. Cases with agreement are accepted.
5. Cases with disagreement or uncertainty are flagged for adjudication.
6. During adjudication, annotators review the raw log, surrounding context, domain information, and related logs.
7. The final label is determined through group discussion.
8. The final adjudicated label is recorded as the ground-truth label for downstream experiments.

5. Parsing-Error Coding Guidelines

5.1 Coding Unit

The coding unit is a parsing-error case. A parsing-error case may consist of:

- A single raw log message and its incorrect parsed template
- A group of similar raw messages incorrectly assigned to one or more templates
- A comparison between the incorrect parsed template and the expected parsing intuition
- A parser output that distorts event identity, variable boundaries, or operational semantics

Each case should be assigned one primary parsing-error type. If multiple error types appear in the same case, annotators should select the error type that best explains the main downstream distortion.

5.2 General Coding Principle

Parsing-error types should be coded based on observable parsing behavior and potential downstream impact, not on the internal implementation of a specific parser.

Annotators should ask:

- Does the parser preserve the operational meaning of the raw log?
- Does the parser separate distinct events that should remain different?
- Does the parser group equivalent events that should share the same template?
- Does the parser correctly identify variable fields?
- Does the parser introduce, remove, or rewrite semantic information?
- Would this parsing output distort event sequences, template frequencies, or anomaly detection inputs?

6. Six Parsing-Error Types

E1: Template Over-Merging

Definition: Distinct log events are incorrectly collapsed into one template.

Operational symptom: The parser masks or abstracts tokens that are semantically important for distinguishing normal and anomalous behavior.

Typical examples:

- Success and failure messages are merged into one template.
- Normal and timeout events are treated as the same event type.
- Different error codes or state transitions are abstracted away.

Coding rule: Use E1 when the main problem is that different event meanings become indistinguishable.

E2: Template Over-Splitting

Definition: Log messages that should share one template are split into multiple templates due to minor lexical, formatting, or parameter variations.

Operational symptom: The event space becomes unnecessarily fragmented, producing sparse and unstable templates.

Typical examples:

- File paths, IDs, IP addresses, or numeric values are treated as constants.
- Minor formatting differences create multiple templates.
- The same operational event is split across software versions.

Coding rule: Use E2 when the main problem is excessive fragmentation of one event type.

E3: Variable Boundary / Extraction Error

Definition: Variable spans are partially, incorrectly, or inconsistently extracted.

Operational symptom: Important identifiers, values, paths, or parameters are corrupted because the parser extracts only part of the variable or includes constant text inside a variable.

Typical examples:

- A request ID is only partially masked.
- A file path is split incorrectly.
- A sensor value and its unit are separated incorrectly.
- A compound identifier is partly treated as constant and partly as variable.

Coding rule: Use E3 when the main problem is incorrect variable boundary detection.

E4: Inconsistent Parameter Masking

Definition: The same type of parameter is treated inconsistently across similar log messages.

Operational symptom: Equivalent values are sometimes masked and sometimes retained as constants, making templates unstable.

Typical examples:

- IP addresses are masked in some messages but retained in others.
- User IDs or transaction IDs are inconsistently abstracted.
- Numeric values are sometimes treated as variables and sometimes as constants.

Coding rule: Use E4 when the same parameter type is handled inconsistently across occurrences.

E5: Unseen Template / OOV Handling Error

Definition: New, rare, or evolving templates are incorrectly matched, collapsed, or assigned because the parser cannot handle unseen log structures.

Operational symptom: Logging drift or rare events lead to incorrect template assignment.

Typical examples:

- A new software-version log is forced into an old template.
- A rare industrial control event is merged with a common event.
- A newly introduced module produces logs that are misclassified into unrelated templates.

Coding rule: Use E5 when the error is mainly caused by new, rare, or previously unseen log structures.

E6: LLM Semantic Rewriting / Hallucinated Template

Definition: An LLM-based parser rewrites, paraphrases, normalizes, or introduces tokens that are not faithfully present in the raw log.

Operational symptom: The generated template becomes semantically plausible but no longer faithfully represents the original log message.

Typical examples:

- The parser paraphrases an operational message.
- The parser inserts explanatory tokens not present in the raw log.
- The parser changes a domain-specific term into a generic term.
- The parser rewrites a warning or failure message in a way that alters its meaning.

Coding rule: Use E6 only for LLM-based parser outputs where semantic rewriting or hallucinated content is the main problem.

7. Priority Rules for Ambiguous Parsing Errors

When multiple parsing errors appear in one case, use the following priority rules:

1. If the parser changes or invents semantic content, assign E6.
2. If distinct operational events are collapsed into one template, assign E1.
3. If one event type is fragmented into many templates, assign E2.
4. If the main issue is incorrect variable span extraction, assign E3.
5. If the main issue is inconsistent masking of the same parameter type, assign E4.
6. If the error mainly arises from a rare, new, or drifting template, assign E5.

Annotators should record secondary error types in a comment field when useful, but only one primary label should be used for quantitative analysis.

8. Parsing Correction Guidelines for RQ2

For corrected parsing experiments, annotators should manually revise only the parsing errors identified through the taxonomy coding process.

Correction should follow these principles:

- Preserve the original raw log semantics.
- Do not add information that is not present in the raw log.
- Do not remove operationally meaningful tokens.

- Keep semantically distinct events as separate templates.
- Merge only messages that express the same operational event.
- Mask dynamic variables consistently across occurrences.
- Keep status codes, failure indicators, control states, and security decisions as constants when they distinguish event meaning.
- Apply correction consistently to both training and test data to avoid distribution mismatch.

Corrected templates should be recorded together with the original parser output and the reason for correction.

9. Reliability Assessment

Before finalizing labels, inter-rater reliability should be assessed between the two expert groups.

For anomaly labeling:

- Compute Cohen's kappa between the two group-level labels.
- Report the kappa value for each dataset or dataset group.
- Values above 0.80 indicate substantial agreement.

For parsing-error coding:

- Compute Cohen's kappa between the two expert groups before adjudication.
- Record the number of agreements, disagreements, and uncertain cases.
- Resolve disagreements through group discussion.
- Use adjudicated labels for final analysis.

10. Required Output Fields

For anomaly labeling, the output file should include:

- Dataset name
- Log ID or sequence ID
- Raw log message or sequence reference
- Context window or session ID, if available
- Group A label
- Group B label
- Final adjudicated label
- Adjudication note, if applicable

For parsing-error coding, the output file should include:

- Dataset name
- Parser name
- Log ID or case ID
- Raw log message
- Incorrect parsed template
- Corrected or expected template
- Group A error type
- Group B error type
- Final error type
- Secondary error type, if applicable

- Correction note
- Adjudication note, if applicable

11. Confidentiality and Anonymization

Annotators must remove or mask sensitive operational identifiers before releasing any representative samples. Sensitive information includes:

- Real user identifiers
- Hostnames or internal IP addresses
- Machine, plant, line, or device identifiers
- Proprietary event codes
- Business transaction IDs
- Access-control records
- Domain-specific configuration details
- Any information subject to confidentiality agreements

Anonymization should preserve the structural and semantic characteristics needed for log parsing and anomaly detection analysis.

12. Quality-Control Checklist

Before finalizing the annotation files, verify that:

- Each case has exactly one final primary label.
- All disagreements have adjudication notes.
- All uncertain labels have been resolved.
- Corrected templates preserve raw-log semantics.
- Dynamic variables are masked consistently.
- Normal/anomalous labels are consistent across similar cases.
- Dataset names, parser names, and case IDs are complete.
- Sensitive identifiers have been anonymized.
- Reliability statistics have been computed and recorded.